



NICE ASSISTANT

Complete Project Book

Project idea, full requirements, implementation plans, architecture, data flows, and designs

Edition: March 2026

Project Root: F:/Hackathon Project

Primary Target: Android first, offline default

Scope	Status
Concept and product vision	Included
PRS and requirement matrix	Included
Implementation plans (original + hardening)	Included
Architecture and module design	Included
Data flow diagrams and system flows	Included
Testing, release, and risk forecast	Included

Table of Contents

1) Project Idea and Vision	3
2) Product Requirement Specification (PRS)	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	3
3) Implementation Plans (Complete)	3
3.1 Original 5-Phase Plan	3
3.2 Hardening and Expansion Plan	3
4) Architecture and Design	4
4.1 Module-Level Design	4
5) Data Flow Diagrams and System Designs	4
5.1 High-Level Architecture (Mermaid)	4
5.2 Index and Search Sequence (Mermaid)	4
5.3 Android Read Fallback (Mermaid)	4
5.4 Voice and Mic Arbitration (Mermaid)	5
5.5 Build and Release Flow (Mermaid)	5
6) Validation and Quality Engineering	5
7) Risk Forecast and Mitigation Design	5
Appendix A - Full Original Implementation Plan (verbatim)	6
Appendix B - Full Original PRD (verbatim)	9
Appendix C - Technical Manuscript (verbatim)	11

1) Project Idea and Vision

Nice Assistant is an offline-capable, privacy-first AI assistant designed to execute useful tasks directly on the device while minimizing cloud dependence. The product philosophy is deterministic local control first, then optional network features under explicit user consent.

- Offline default behavior for core assistant tasks and local knowledge search.
- Natural language interface for files, apps, timers, notes, and system actions.
- On-device intelligence stack with worker-based heavy processing to keep UI responsive.
- Android-first execution model with sideload release quality and strong fallback design.

2) Product Requirement Specification (PRS)

2.1 Functional Requirements

ID	Requirement	Design intent
FR-01	Intent parser and deterministic routing	Predictable control over device tasks
FR-02	Deep file indexing and scan	High-coverage local corpus for offline answers
FR-03	Text extraction (TXT/PDF/IMG/DOCX)	Search inside file content, not only names
FR-04	Hybrid retrieval (BM25 + rerank)	Better relevance with low latency
FR-05	Wake word + STT + TTS	Hands-free conversational operation
FR-06	Automation rules	User-defined local trigger-action workflows
FR-07	Floating bubble and timer bubble	Cross-app accessibility and live timer visibility
FR-08	Policy-gated online mode	User-controlled connectivity with clear boundaries
FR-09	Encrypted personalization memory	Local adaptation with privacy guarantees
FR-10	Multimodal OCR flow	Image/screenshot understanding for local Q/A

2.2 Non-Functional Requirements

- Privacy: offline default and explicit online session opt-in.
- Reliability: robust fallback chain for Android scoped-storage constraints.
- Performance: low-end profile support and throttled maintenance loops.
- Security: endpoint allowlisting and encrypted local memory storage.
- Maintainability: modular source files and release gate automation.

3) Implementation Plans (Complete)

3.1 Original 5-Phase Plan

- Phase 1 - Foundation and setup (Vite + Capacitor + UI + permissions).
- Phase 2 - Core intents and command parser with TDD.
- Phase 3 - Offline AI and retrieval engine with worker-based processing.
- Phase 4 - Native Android indexing and extraction pipelines.
- Phase 5 - Testing, optimization, and signed APK release.

3.2 Hardening and Expansion Plan

- Runtime policy hardening and truthful response guarantees.
- Content-first retrieval ordering and full-scan behavior.
- Mic arbitration and voice lifecycle stabilization.

- Incremental indexing maintenance and stale-index recovery.
- Knowledge graph, answer guard, and confidence-aware fallback.
- Performance governor and proactive suggestion systems.
- Release gating and signed build quality controls.

4) Architecture and Design

4.1 Module-Level Design

Layer	Files	Description
UI and Orchestration	src/main.js, index.html, src/style.css	Conversation UI, settings, voice control, and runtime hooks
Assistant Core	src/assistant.js, src/offline-brain.js	Parser, intent generation, action routing, and planning
Search and Knowledge	src/search-engine.js, src/knowledge-graph.js	Extraction, ranking, synthesis, and evidence memory
Native and Storage	src/filesys.js, IndexedDB, localStorage	Index lifecycle, file access, caching, and metadata
Voice and Automation	src/wake-word.js, src/mic-arbiter.js, src/automation-engine.js	Wake-word, safety and trigger-action execution
Security and Policy	src/policy.js, src/secure-store.js, src/usage-leads.js	Privacy control and encrypted personalization state
Performance Controls	src/performance-governor.js, src/proactive-engagement.js	Adaptive runtime behavior and user guidance

5) Data Flow Diagrams and System Designs

5.1 High-Level Architecture (Mermaid)

```
graph TD
    User([User Voice/Text]) --> UI[Frontend Chat Interface]
    UI --> IntentParser{Intent Parser}
    IntentParser --> NativeBridge[Capacitor Native Bridge]
    NativeBridge --> AndroidOS[Android Services]
    IntentParser --> Retrieval[Local Retrieval Engine]
    Retrieval --> FileIndex[(IndexedDB File Index)]
    Retrieval --> EmbedWorker[[Embedding Worker]]
    EmbedWorker --> Rerank[Semantic Rerank]
    Rerank --> Guard[Answer Guard]
    Guard --> UI
    UI --> TTS[Speech Output]
```

5.2 Index and Search Sequence (Mermaid)

```
sequenceDiagram
    participant U as User
    participant FS as File Scanner
    participant SE as Search Engine
    participant EX as Extractors
    participant DB as IndexedDB

    FS->>DB: Index metadata and paths
    U->>SE: Ask question
    SE->>DB: Load readable files
    SE->>EX: Extract text/OCR as needed
    EX-->>SE: Text chunks
    SE->>SE: BM25 rank + embedding rerank
    SE-->>U: Contextual answer with evidence
```

5.3 Android Read Fallback (Mermaid)

```
flowchart TD
    A[Read request] --> B[Filesystem.readFile]
    B -->|success| C[Parse content]
    B -->|failure| D[Filesystem.getUri]
    D --> E[convertFileSrc + fetch]
    E -->|success| C
    E -->|failure| F[Filename fallback]
    F --> G[Partial response]
    C --> G
```

5.4 Voice and Mic Arbitration (Mermaid)

```
stateDiagram-v2
    [*] --> IDLE
    IDLE --> WAKE_WORD: Enable wake word
    WAKE_WORD --> LISTENING: Trigger detected
    LISTENING --> PROCESSING: STT complete
    PROCESSING --> IDLE: Response emitted
    LISTENING --> IDLE: Timeout/Cancel
```

5.5 Build and Release Flow (Mermaid)

```
flowchart LR
    SRC[Source code] --> TEST[release:check]
    TEST --> WEB[vite build]
    WEB --> SYNC[cap sync android]
    SYNC --> GRADLE[assembleRelease]
    GRADLE --> SIGNVERIFY[apksigner verify]
    SIGNVERIFY --> APK[Signed APK]
```

6) Validation and Quality Engineering

- Release gate validates required model assets and all hardening test suites.
- Regression suites enforce parser integrity, policy behavior, retrieval ordering, and worker lifecycle reliability.
- Deployment checks verify package installability, launchability, and runtime process health on-device.

7) Risk Forecast and Mitigation Design

- Storage permission variance across OEM builds: explicit in-app state labels and fallback messaging.
- Large corpus search latency: adaptive performance profile and staged progress callbacks.
- Answer quality regressions: confidence guard + clarification prompts + evidence-centric synthesis.
- Feature drift risk: dedicated stability forecast tests and duplicate-path removals.

Appendix A - Full Original Implementation Plan (verbatim)

Project Implementation Plan: "Nice" Offline AI Assistant

1. Project Overview & Description

"Nice" is a privacy-first, fully offline AI Assistant application designed primarily for Android devices (via Capacitor) and desktop web browsers. Unlike traditional AI assistants that send all user data to the cloud, Nice runs large language models (LLMs), Wake Word detection, Vector Embeddings, and Optical Character Recognition (OCR) directly on the device's edge hardware using WebGPU and WebAssembly.

Its primary goal is to provide a fully transparent, highly responsive, and data-secure conversational interface capable of interacting with the device's native ecosystem (files, calendar, apps, alarms) without an internet connection.

2. Core Functionalities

The application is divided into several modules of functionality:

2.1 Natural Language Processing & Local AI

- **On-Device LLM Inference:** Uses ONNX Runtime Web and WebGPU to run a local language model for chatting and understanding context.
- **Wake Word Detection:** Continuously listens for a specific trigger phrase (e.g., "Hey Nice") using an efficient, lightweight local audio-classification model.
- **RAG (Retrieval-Augmented Generation):** Embeds local documents and knowledge using Transformer-based feature extraction (`@xenova/transformers`) and stores them in a local Vector Database (IndexedDB) for answering context-specific questions.

2.2 Device & OS Integration

- **Deep File Search (Native):** Recursively indexes the entire Android external storage, supporting over 50 file types (PDF, Word, TXT, Excel, Code). Extracts text and OCRs images.
- **Intent Parsing System:** A regex and rule-based fallback system that can route immediate commands:
 - App Launching (e.g., "Open Spotify")
 - Alarm & Calendar Management
 - Media Controls ("Play Bohemian Rhapsody")
 - Weather parsing and mathematical expression evaluation (including "word math" like "two times four").

2.3 Audio & Accessibility

- **Speech-to-Text (STT):** Local transcription or Whisper-based models.
- **Text-to-Speech (TTS):** Uses the native Android TTS engine or Web Speech API to read responses aloud.
- **Floating Chat Head:** An overlay service (Service Window) that allows the assistant to be summoned from anywhere in the OS.

3. Technology Stack

Category	Technology	Purpose
Frontend	Vanilla HTML/CSS/JS, Vite	High performance, lightweight DOM manipulation without massive framework overhead.
Mobile Runtime	Capacitor (v8)	Wraps the web app into a native Android APK, providing bridges to native device features.
Local AI Engine	ONNX Runtime Web, <code>@xenova/transformers</code>	Runs AI models (LLMs, Whisper, Embeddings) locally using Wasm/WebGPU.
Text Extraction	<code>pdfjs-dist</code> , <code>tesseract.js</code> , <code>mammoth</code>	Extracts raw text from PDFs, Images (OCR), and Word Documents respectively.
Database	IndexedDB	Local storage of file metadata, text chunks, and vector embeddings for RAG.

4. Software Development Lifecycle (Step-by-Step Approach)

Building this from scratch follows a 5-phase approach.

Phase 1: Foundation & Project Setup

1. **Initialize Project:** Create Vite vanilla JS template.
2. **Capacitor Integration:** Install `@capacitor/core`, `@capacitor/android`, and initialize the `[capacitor.config.json](file:///f:/Hackathon%20Project/capacitor.config.json)`.
3. **UI Prototyping:** Build the chat interface, microphone button, and settings menu. Implement CSS variables for dark/light themes.
4. **Permissions Architecture:** Implement a startup sequence that requests all necessary Android permissions (`RECORD_AUDIO`, `MANAGE_EXTERNAL_STORAGE`, `SYSTEM_ALERT_WINDOW`).

Phase 2: Core Assistant Logic & Intents

1. **Command Parser:** Build `[test-all.mjs](file:///f:/Hackathon%20Project/tests/test-all.mjs)` first (TDD

approach). Implement Regex-based extraction modules for Alarms, Calendars, App Launching, and Math.

- Capacitor Plugins Integration:** Use the `@capacitor/app-launcher` to trigger Android intents for opening apps and setting alarms via standard URIs.
- Speech Integration:** Wire up the `window.SpeechRecognition` (or polyfill) for listening to the user, and `window.speechSynthesis` for the assistant's voice.

Phase 3: The Offline AI & RAG Engine

- Model Bundling:** Create scripts (`[download-model.mjs](file:///f:/Hackathon%20Project/download-model.mjs)`, `[download-tess.mjs](file:///f:/Hackathon%20Project/download-tess.mjs)`) to pull required ONNX models locally into the `public/` directory so the APK ships with them.
- Web Worker Threading:** Build `[worker.js](file:///f:/Hackathon%20Project/src/clip-worker.js)` to offload the heavy WebGPU LLM inference and embedding calculations so the main UI thread never freezes.
- Vector Database:** Build a minimalist class wrapping IndexedDB to store 384-dimensional arrays (embeddings) and do Cosine Similarity matching for searches.

Phase 4: Native Android Capabilities & Indexing

- File System Scanner** (`[filesystem.js](file:///f:/Hackathon%20Project/src/filesystem.js)`): Implement the recursive scanner utilizing `@capacitor/filesystem`. Handle Android 11+ Scoped Storage by falling back to `FileSystem.getUri` + `fetch` for reading file bytes.
- Text Extraction Pipeline** (`[search-engine.js](file:///f:/Hackathon%20Project/src/search-engine.js)`): Integrate Tesseract for `.png/.jpg`, `PDF.js` for `.pdf`. Slice extracted text into paragraphs.
- Background Services:** Implement the Android `Service` wrapper in Java to support the floating chat head overlay, keeping the WebView alive in the background.

Phase 5: Testing, Optimization, and Release

- Performance Profiling:** Optimize memory limits. Throttle the file scanner (e.g., yielding every 30ms) to avoid "App Not Responding" (ANR) crashes on low-end phones.
- Testing suite:** Expand the TDD suite to cover edge cases like division by zero in word math, or missing file extensions.
- APK Build:** Run Android Studio/Gradle to generate the signed `release.apk` mapping `targetSdkVersion 34`.

5. Data Flow & Architecture Diagrams

5.1 High-Level Architecture

```

```mermaid
graph TD
 User([User Voice/Text]) --> UI[Frontend Chat Interface]
 UI --> IP[IntentParser{Intent Parser}]

 IP -- "Launch App, Alarm, Math" --> NB[NativeBridge[Capacitor Native Bridge]]
 NB --> OS[OS[Android OS Services]]

 IP -- "General Query" --> RAG[RAG[RAG & Search Engine]]
 IP -- "File Search" --> RAG

 RAG --> FS[FileSys[Local Storage Scanner]]
 FS --> VDB[VectorDB[(IndexedDB Vectors & Meta)]]

 RAG --> AIW[AIWorker[[AI Web Worker Thread]]]
 AIW --> ONNX[ONNX[ONNX WebGPU Runtime]]
 ONNX --> LLM[LLM[(Local LLM Model)]]

 ONNX --> RG[ResponseGen[Generate Response]]
 RG --> UI
 UI --> TTS[TTS[Text-to-Speech Engine]]
 TTS --> User
```

```

5.2 File Indexing & Search RAG Data Flow

```

```mermaid
sequenceDiagram
 participant U as User
 participant SE as Search Engine
 participant FS as File System
 participant EXT as Extractors (PDF/OCR)
 participant VDB as Vector DB / Index
 participant LLM as Local LLM

 %% Indexing Phase
 Note over SE, VDB: 1. Background Indexing Phase
 FS->>FS: Scan entire Android Root Storage

```

```

FS-->SE: Yield File List (Text, PDFs, Images)
SE-->EXT: Send raw bytes for unsupported formats
EXT-->SE: Return raw text strings
SE-->SE: Chunk text into paragraphs
SE-->VDB: Store Metadata + Text Chunks

%% Retrieval Phase
Note over U, LLM: 2. Active Search & Retrieval
U-->SE: "What is my March invoice total?"
SE-->SE: Tokenize Query ("March", "invoice", "total")
SE-->VDB: Query matching paragraphs (BM25 or Cosine)
VDB-->SE: Return top 5 relevant chunks
SE-->LLM: Prompt Context: [Chunks] + "Answer the user."
LLM-->U: "Your March invoice total is $3,847.50."
...

```

### ### 5.3 Error Handling & Fallback Strategy (Android 11+)

```

```mermaid
flowchart TD
    Start[Request File Read] --> CapRead[Try Filesystem.readFile]
    CapRead -- Success --> Parse[Extract Text]
    CapRead -- Failure --> GetURI[Try Filesystem.getUri]
    GetURI -- Return URI --> Convert[Capacitor.convertFileSrc]
    Convert --> Fetch[Try fetch Web URL]
    Fetch -- Success --> Parse
    Fetch -- Failure --> NameFallback[Match by Filename Only]
    NameFallback --> Display[Return Partial Search Result]
    Parse --> Display
...
---

```

Summary

Building "Nice" requires a careful balance between modern Web AI (ONNX, Transformers.js) and restrictive Android Native APIs (Scoped Storage, Background Services). By keeping the ML inference inside Web Workers and building robust error catchers/fallbacks for the file system bridge, the application achieves a highly responsive, entirely private assistant experience.

Appendix B - Full Original PRD (verbatim)

Product Requirements Document: Nice Offline Mobile Assistant

****Version**:** 1.0.0
****Status**:** Draft
****Owner**:** AI Assistant Team

1. Executive Summary

The ****Nice Offline Mobile Assistant**** is a privacy-first, high-performance personal assistant designed to operate entirely on-device. By leveraging Small Language Models (SLMs) and modern WebAssembly-based AI, it eliminates the need for cloud connectivity, ensuring data remains private and the assistant remains functional even in "off-grid" scenarios. It bridges the gap between web-based AI and native mobile automation.

2. Target Audience

- ****Privacy Enthusiasts**:** Users who do not want their data sent to cloud servers for processing.
- ****Low-Connectivity Users**:** Individuals in areas with unreliable internet who still need intelligent assistance.
- ****Power Users**:** Users seeking fast, local file search and device automation without cloud latency.
- ****Security-Conscious Organizations**:** Entities requiring offline AI for confidential data handling.

3. User Stories

- ***As a user in a remote area*, I want to ask my assistant for information from my local PDFs without an internet connection.**
- ***As a privacy-focused user*, I want to use voice commands to set alarms and play music without my voice being recorded on a server.**
- ***As a student*, I want to quickly search through hundreds of textbook pages for a specific concept using natural language.**
- ***As a professional*, I want to take voice notes and have them transcribed instantly while traveling.**

4. Functional Requirements

4.1 Voice Interaction & AI

- ****Wake Word Detection**:** Identify a specific trigger word to activate the assistant.
- ****Offline speech-to-Text (STT)**:** Transcribe user speech using `@xenova/transformers` (Whisper) completely offline.
- ****Text-to-Speech (TTS)**:** Provide spoken responses using native system voices.
- ****Local SLM**:** Process natural language queries using `@mlc-ai/web-llm` (WebLLM) on the device's CPU/GPU.
- ****Contextual Memory**:** Maintain a short-term chat history for follow-up questions.

4.2 Intelligence & File Parsing

- ****Deep-File Search**:** Index and search local PDF, Docx, and Text files.
- ****Image OCR**:** Extract text from images and screenshots using `Tesseract.js`.
- ****Sliding Window Scoring**:** Use advanced ranking (BM25-lite) to find precise "pinpoint" answers within large documents.
- ****File Watcher**:** Automatically detect and index new or modified files in specific directories.

4.3 Device Automation

- ****App Launcher**:** Open installed Android applications (Spotify, Maps, Calculator, etc.).
- ****Media Control**:** Play, pause, and search for local audio files.
- ****Alarms & Timers**:** Set, cancel, and manage system-level alarms and countdown timers.
- ****Calendar & Flashlight**:** Native integration for scheduling and hardware controls (vibration, flashlight).

4.4 Utility Features

- ****Smart Math**:** Handle complex math/percentage calculations via natural language (e.g., "What is 15% of 850?").
- ****Local Notes**:** Create, view, and delete notes persisted in `localStorage`.
- ****Battery & System Info**:** Report current device status (battery level, storage stats).

5. Non-Functional Requirements

- ****Privacy**:** Zero data transmission to external servers.
- ****Performance**:**
 - Search response time < 100ms.
 - AI inference time optimized for mobile hardware (WebGPU/WASM).
- ****Efficiency**:** Memory management logic to prevent crashes on 4GB RAM devices (5MB hardware cap for large file chunks).
- ****Offline-First**:** All core models (Whisper, LLM) must be bundle-able or downloadable for permanent offline use.

6. Technical Architecture

- ****Frontend**:** Vite + Vanilla JavaScript + CSS (Glassmorphism design).
- ****Native Bridge**:** Capacitor (coupling web views with Android hardware APIs).

- **AI Engine**:
 - WebLLM (MLC-AI) for the Language Model.
 - Transformers.js (ONNX) for Whisper STT.
- **Data Layer**:
 - **IndexedDB**: Persistent file index and metadata.
 - **LocalStorage**: User settings, chat history, and notes.
 - **Filesystem API**: Native read/write access via `capacitor-file-system`.

7. UI/UX Vision

- **Modern Aesthetic**: Dark mode support, smooth CSS animations, and a "vibrant" premium feel.
- **Accessibility**: Clear visual indicators for listening/processing states.
- **Responsiveness**: Mobile-first design that scales to tablets and desktops.

8. Future Roadmap

- **Vision Integration**: Local image classification and object detection.
- **Plugin System**: Allow users to add custom automation scripts.
- **Improved RAG**: Multi-modal search (searching images by content descriptions).
- **Smart Home Integration**: Local control for IoT devices (Matter/Thread support).

Appendix C - Technical Manuscript (verbatim)

The Nice Offline Mobile Assistant: A Comprehensive Project Manuscript

1. Introduction & Philosophy

The **Nice Assistant** was born from a fundamental critique of modern computing: *Why must we rely on cloud-connected AI, surrendering our privacy, battery life, and internet bandwidth, just to perform manual, on-device tasks?*

This project was built to transform a standard mobile phone into a self-sufficient intelligence hub. The vision was to create a fiercely private, blazingly fast personal assistant that lives exclusively on the "Edge" (the local device CPU/RAM) and operates entirely without an internet connection. It bridges the gap between raw hardware capabilities and natural language understanding.

2. Core Capabilities & Use Cases

The assistant is engineered to automate tasks that are typically tedious for users, specifically targeting offline capabilities:

- Intelligent Deep-File Parsing**: Breaking down monolithic PDFs, Image OCRs (via Tesseract.js), and extensionless text documents to instantly search, summarize, and extract precise information without uploading a single byte to the cloud.
- Device Automation**: Natively hooking into the Android OS to launch installed applications (like Spotify, Maps, or Calculator) or opening the Play Store if the app isn't found.
- Local Media Playback**: Accessing local storage to play audio files directly within the chat interface.
- On-Device Voice Recognition**: Transcribing spoken audio completely offline using the `@xenova/transformers` ONNX 'Whisper'`` machine learning model.
- Utilitarian Action Engine**: Handling complex math/percentage calculations natively via a JavaScript "Word Math" parser, setting alarms/timers, and persisting user notes.

3. The Technical Architecture

The Nice Assistant operates on a lightweight, web-native stack bridged directly to device hardware.

- Frontend**: Built with Vanilla JavaScript, HTML, and CSS (featuring a modern Glassmorphism design system), bundled via Vite for extreme compression and speed.
- Native Bridge**: Powered by **Capacitor**, which wraps the web application into a native Android container, granting it direct access to the filesystem, intents, UI interactions, and background processes.
- AI Engine**: Integrates `Transformers.js` (ONNX)`` to utilize the device's WebAssembly (WASM) capabilities for offline Speech-to-Text (Whisper).
- Data Layer**:
 - IndexedDB**: Maintains a persistent file index and metadata, allowing rapid re-hydration of the searchable corpus on reboot.
 - LocalStorage**: Manages user settings, active alarms, and chat history.

4. Engineering Challenges & Android Optimizations

Building a truly "offline-first" application capable of processing gigabytes of local data required overcoming significant engineering hurdles, particularly concerning Android fragmentation and strict hardware limits (like a 4GB RAM ceiling).

4.1 Memory Management & Crash Prevention

Mobile WebViews (which power Capacitor apps) frequently suffer from "Out of Memory" (OOM) crashes when attempting to read or index massive file payloads.

- Extension Filtering**: The engine was rigorously restricted to only index specific, readable file types (`.txt`, .md`, .pdf`, .jpg`, .png``). It actively ignores heavy application files (like `.js` or .dll``) that would instantly bloat the RAM.
- Hardware Cap & Chunking**: We engineered a 5MB hardware cap per file reading operation and implemented batched processing (scanning a maximum of 15 files concurrently), manually yielding the main thread to prevent the UI from freezing.

4.2 The "Instant-On" Search Algorithm

Instead of a slow, brute-force file sweep or relying on heavy Vector embeddings, we built a highly optimized Inverted Index (BM25-lite) that operates in near `O(1)`` memory lookup time.

- Sliding Window Scoring**: The engine evaluates the true density of paragraphs rather than just substring matches, returning precise, perfectly formatted answers in under 100 milliseconds.
- Caching Strategy**: A two-tiered caching system maintains a 60-second TTL for raw file text to accelerate follow-up questions, and a 2-minute TTL for identical query results.

4.3 Navigating Android's Privacy Architecture (Scoped Storage)

One of the most significant challenges was Android 13's strict "Scoped Storage" paradigm. Older Android versions allowed apps to request `MANAGE_EXTERNAL_STORAGE`` to scan the entire root directory. Android 13 silently blocks these background scans, resulting in "0 files found" errors.

- The Seamless Fallback**: To bypass this without requiring complex OS settings navigation from the user, we implemented a native HTML File Picker fallback (`_grantViaFilePickerFallback``).
- How it Works**: If the automated background scan is blocked, the app instantly opens the native Android document picker. The user taps the specific PDFs they want to inject into the assistant's brain. This grants

instant, secure, and permanent read access without triggering any alarming permission prompts, perfectly respecting Android's privacy design.

4.4 APK Bloat Reduction

Initial iterations experimented with integrating full Large Language Models (LLMs) via WebGPU. While powerful, the cached model weights (e.g., `SmolLM2-135M`) bloated the APK size by over 500MB.

* **The Pivot**: To maintain a lightweight footprint suitable for devices with constrained storage (like a 128GB Realme 8), the WebLLM dependency and its heavy cached weights were entirely purged from the `public/models` directory. The assistant reverted to its ultra-fast, pattern-matching and deterministic logic engine (`assistant.js`), shedding the bloat and compiling down to a lean, efficient Native Android APK in seconds.

5. Conclusion

The project culminated in a robust, lightweight, and genuinely privacy-first mobile application. The Nice Assistant successfully proves that intelligent automation and deep file parsing do not require a connection to the cloud. By leveraging modern WebAssembly libraries, clever indexing algorithms, and direct hardware API bridging via Capacitor, the assistant delivers a premium, instant user experience-completely off the grid.